

¿Qué es la Ingeniería del Software?

La Ingeniería del Software es la disciplina que aplica principios de ingeniería para diseñar, desarrollar, probar, mantener y gestionar sistemas de software de forma organizada, eficiente y con calidad.

En otras palabras, no se trata solo de programar: consiste en construir software confiable siguiendo un proceso planificado.

Definición sencilla

La Ingeniería del Software es el conjunto de métodos, herramientas y buenas prácticas para crear software que satisfaga las necesidades de los usuarios, dentro de un tiempo y presupuesto determinados.

¿Qué es un Ingeniero de Software?

Un **Ingeniero de Software** es un profesional que utiliza conocimientos de **programación, ingeniería, diseño y gestión** para crear y mantener sistemas de software de manera **organizada, segura, eficiente y confiable**.

No se limita a escribir código: participa en prácticamente todo el **ciclo de vida del software**.

¿Qué tareas puede realizar?

Un ingeniero de software puede:

1.  **Analizar requisitos**
Determinar qué necesita el usuario o la empresa.
2.  **Diseñar sistemas**
Definir la arquitectura, componentes, bases de datos y comunicación entre sistemas.
3.  **Programar**
Desarrollar aplicaciones utilizando lenguajes como Java, Python, C#, JavaScript, etc.
4.  **Realizar pruebas**
Detectar errores y comprobar que el software cumple los requisitos.
5.  **Implementar y desplegar**
Llevar las aplicaciones a servidores, nubes o dispositivos de los usuarios.
6.  **Mantener y evolucionar el software**
Corregir errores, mejorar el rendimiento y agregar nuevas funcionalidades.
7.  **Trabajar en equipos**
Colaborar con analistas, programadores, testers, diseñadores, administradores de sistemas y clientes.

Programador vs. Ingeniero de Software

Una forma sencilla de explicarlo a estudiantes sería:

El programador se concentra principalmente en escribir código; el ingeniero de software se ocupa de que todo el sistema de software sea correctamente diseñado, construido, probado, implementado y mantenido.

Por ejemplo, para desarrollar un **sistema de gestión escolar**, el ingeniero de software podría participar en:

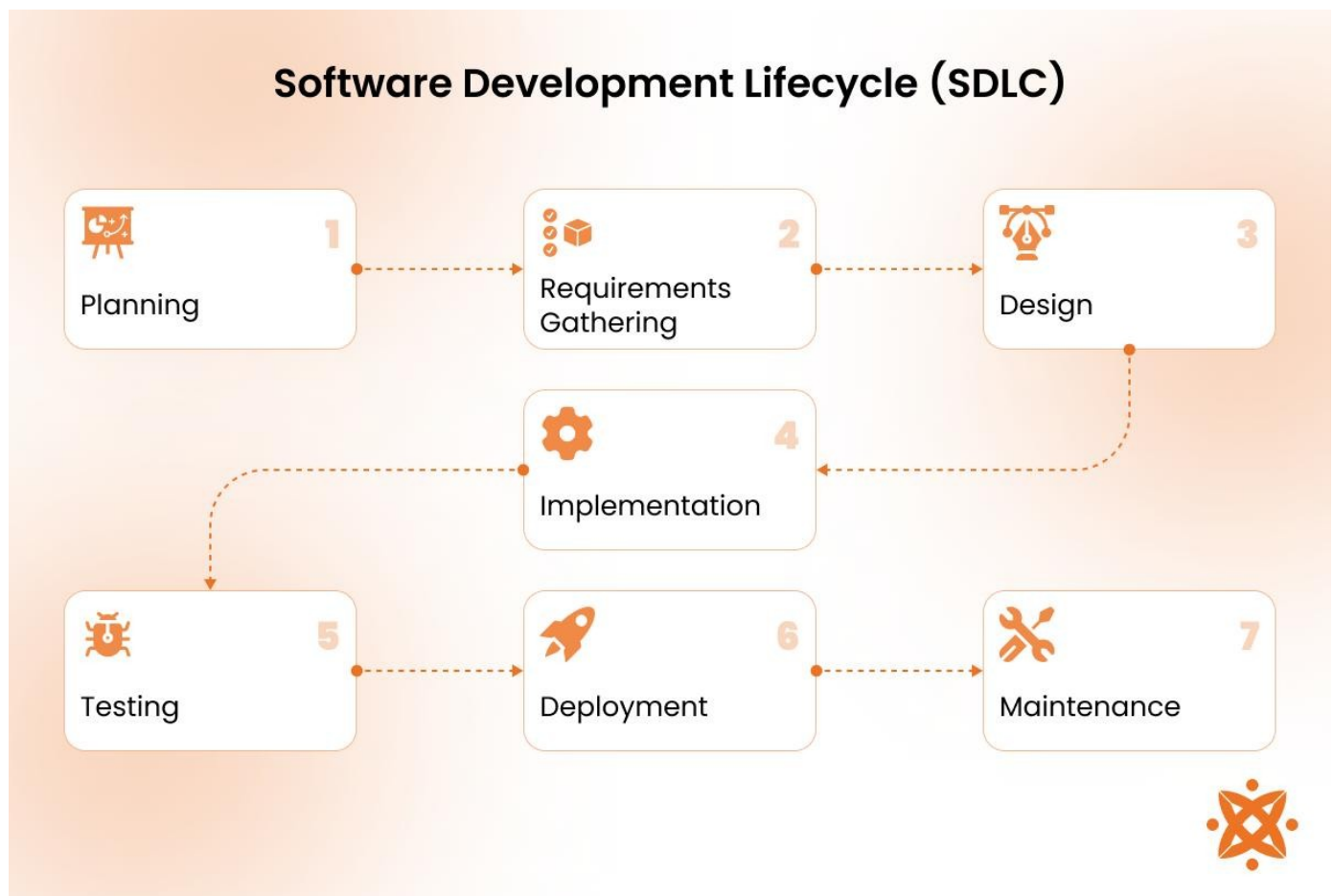
Necesidad → Requisitos → Diseño → Base de datos → Programación → Testing → Implementación → Mantenimiento

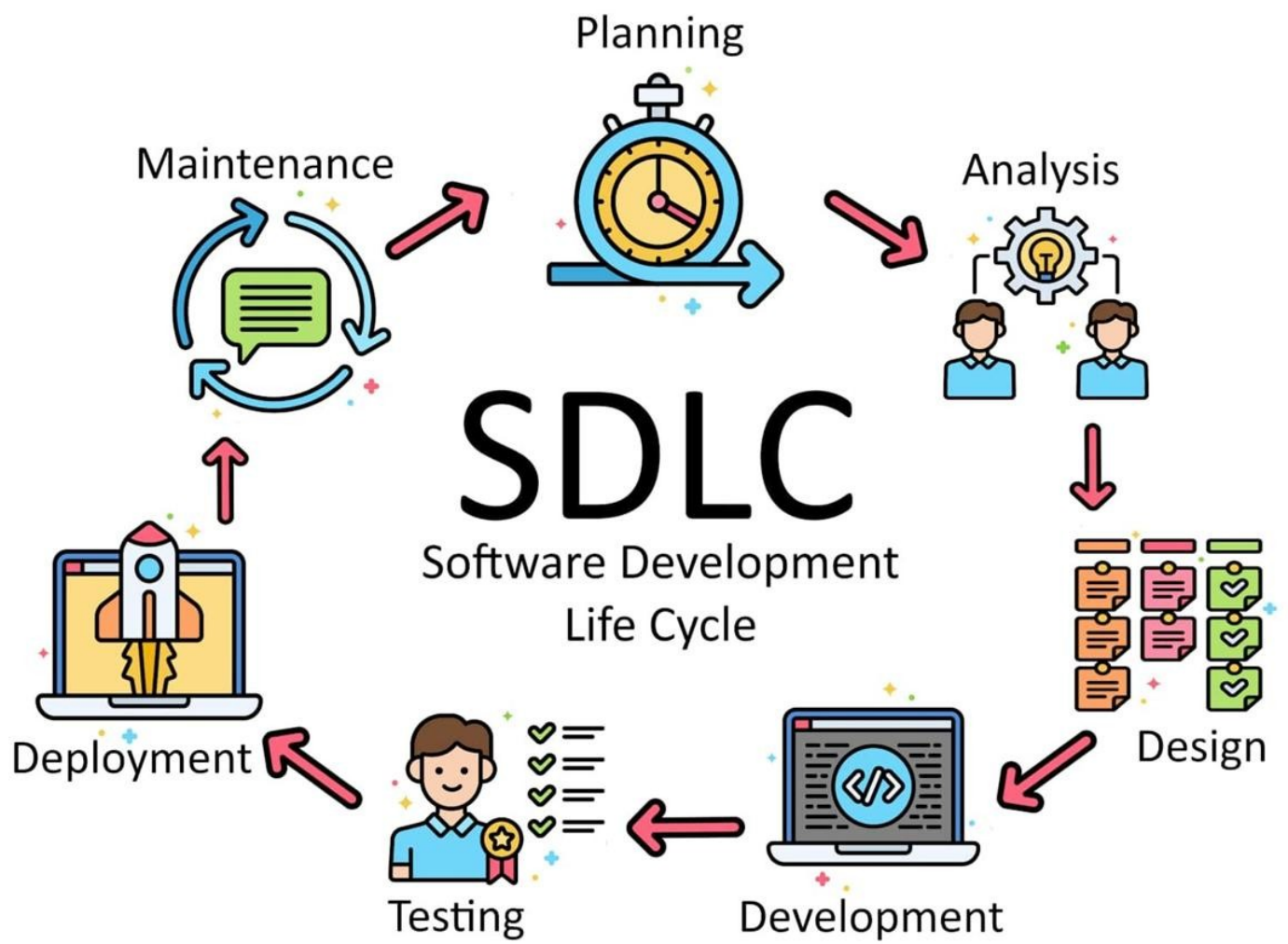
Ciclo de vida del software

El **ciclo de vida del software** es el conjunto de **etapas por las que pasa un sistema de software desde que surge la necesidad de crearlo hasta que deja de utilizarse**.

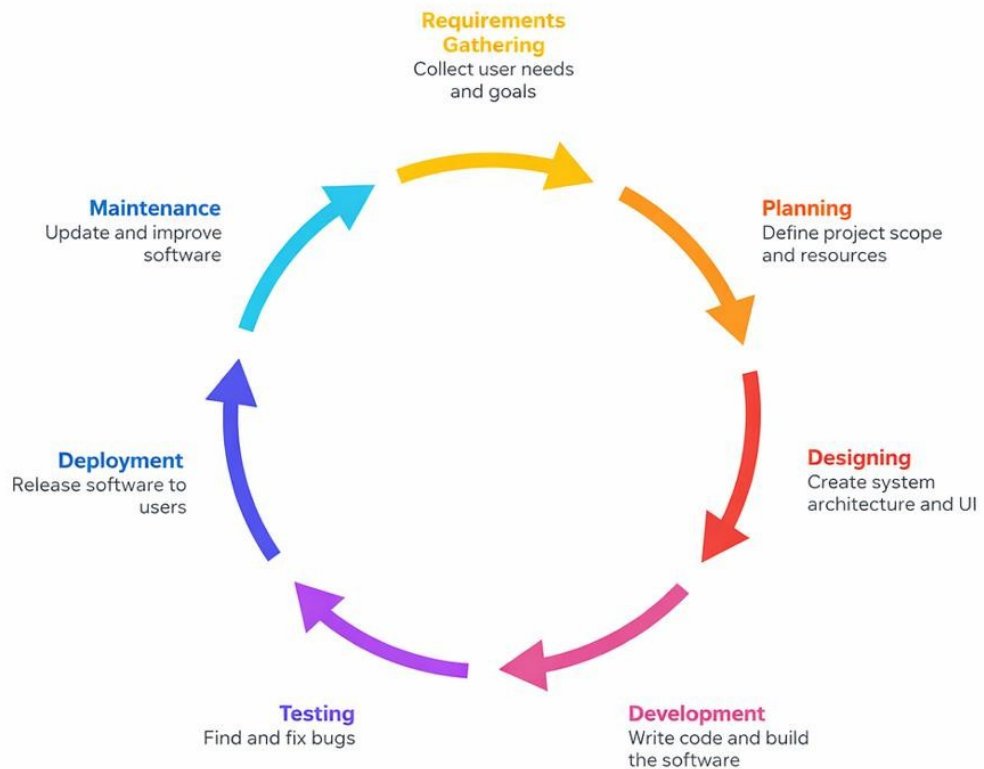
Una forma sencilla de verlo es:

Necesidad → Requisitos → Diseño → Desarrollo → Pruebas → Implementación → Mantenimiento → Retiro





Software Development Life Cycle



5

1. 📋 Análisis y requisitos

Se determina **qué debe hacer el sistema**.

Se recopila información de los usuarios y clientes.

Ejemplo: una escuela necesita un sistema para registrar alumnos, cursos y calificaciones.

Se definen requisitos como:

- Registrar alumnos.
- Modificar datos.
- Registrar calificaciones.
- Generar informes.

2. 🏗️ Diseño

Se decide **cómo se va a construir el sistema**.

Se define, por ejemplo:

- Arquitectura del sistema.
- Base de datos.
- Interfaces.
- Componentes.

- Tecnologías que se utilizarán.

Por ejemplo:

Frontend → API → Backend → Base de datos

3. **Desarrollo / Implementación**

Los programadores **escriben el código** y construyen el software de acuerdo con el diseño.

Aquí aparecen actividades como:

- Programación.
 - Control de versiones.
 - Integración de componentes.
 - Revisiones de código.
-

4. **Pruebas**

Se verifica que el software **funcione correctamente y cumpla los requisitos**.

Se pueden realizar:

- Pruebas unitarias.
- Pruebas de integración.
- Pruebas funcionales.
- Pruebas de sistema.
- Pruebas de aceptación.
- Pruebas de seguridad y rendimiento.

Por ejemplo:

¿Qué ocurre si intento registrar un alumno sin ingresar su DNI?

5. **Implementación / Despliegue**

El software se instala y se pone **a disposición de los usuarios**.

Puede ser:

- Instalado en una PC.
 - Instalado en un servidor.
 - Publicado en Internet.
 - Implementado en la nube.
 - Distribuido como aplicación móvil.
-

6. **Mantenimiento**

Una vez que el software está funcionando, comienza una etapa que puede durar **años**.

Se realizan:

- Correcciones de errores.
- Actualizaciones.
- Mejoras de rendimiento.
- Nuevas funcionalidades.
- Adaptaciones a nuevos sistemas operativos o tecnologías.

Por ejemplo, si cambia una normativa educativa, habrá que modificar el sistema.

7. Retiro

Finalmente, el software puede quedar **obsoleto o ser reemplazado**.

Se realizan actividades como:

- Migración de datos.
 - Desinstalación.
 - Reemplazo por un nuevo sistema.
 - Archivado de información.
-

Un punto muy importante

Estas etapas **no necesariamente ocurren una sola vez y en línea recta**.

En los modelos modernos, especialmente en **Agile**, se repiten continuamente:

Planificar → Desarrollar → Probar → Entregar → Recibir feedback → Mejorar → Volver a desarrollar

Por eso podemos pensar en dos grandes formas de entender el ciclo de vida:

Modelo tradicional	Modelo ágil
Etapas más secuenciales	Iteraciones frecuentes
Mucha planificación inicial	Planificación continua
Entrega generalmente al final	Entregas incrementales
Cambios más costosos	Adaptación al cambio
Ej.: Cascada	Ej.: Scrum, Kanban

Una definición para estudiantes

El ciclo de vida del software es el conjunto de etapas que permiten transformar una necesidad en un producto de software, desde la identificación de los requisitos hasta su mantenimiento y eventual retiro.

Y hay una distinción importante: **el ciclo de vida describe qué actividades se realizan; el modelo de desarrollo define cómo se organizan esas actividades**. Por ejemplo, **Cascada, Iterativo, Incremental y Agile** son diferentes formas de organizar el ciclo de vida.